Class: Algorithms & Data Structures Semester: Spring 2025

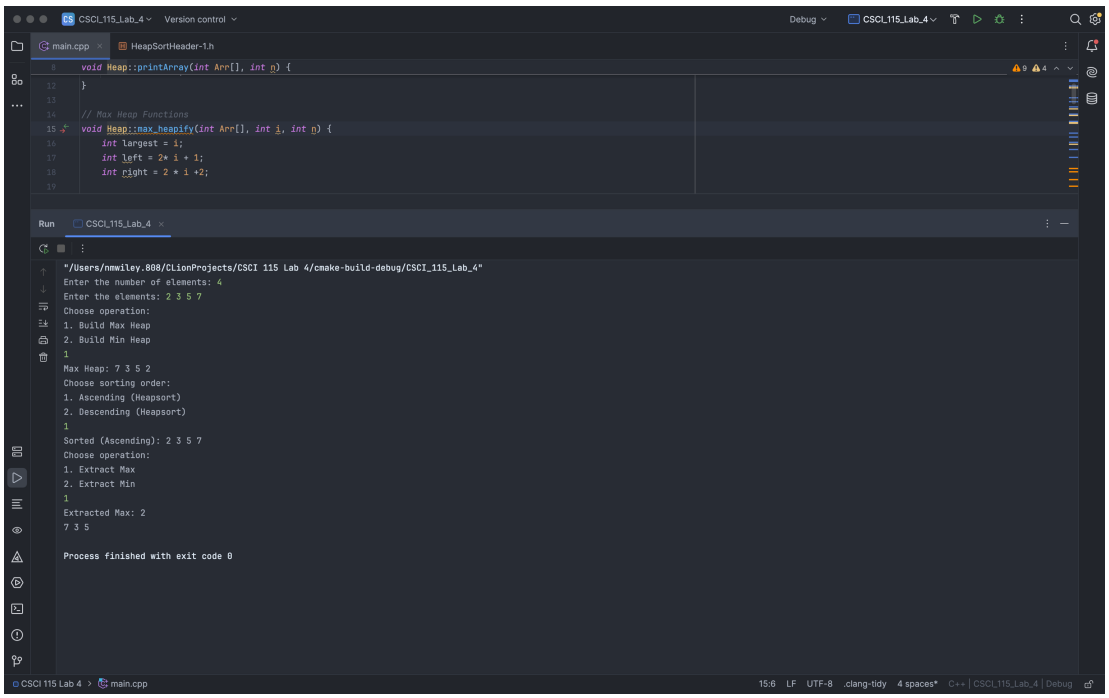
Noah Wiley
noahwiley@mail.fresnostate.edu
Lab Number - Section 09

CSCI 115 Lab 4 Report; Heap Sort

- Statement of Objectives
  - The object of this lab is to implement a Priority Queue ADT using Binary Heaps. This involves constructing both Max Heap and Min Heap, performing Heap Sort in ascending or descending order, and allowing operations such as inserting elements, modifying values, and extracting minimum or maximum elements. This implementation demonstrates the efficiency of binary heaps for priority queue operations. This report details the experimental procedure, analysis, encountered challenges, and conclusions drawn from this experiment.

- Experimental Procedure
  - Accept a list of numbers as input from the user
  - Allow the user to choose between building a Max or Min Heap
  - Implement Heap Sort to order the last in ascending or descending order
  - Provide functionally to extract the minimum or maximum element from the heap
  - Allow the user to modify values in the heap
  - Enable the user to insert new elements into the heap dynamically
  - Print the heap at each stage to visualize transformations

- Analysis
  - The program successfully implemented the Priority Queue ADT using Binary Heaps. Below are the key results:
    - Heap Construction: The program correctly converted an unsorted array into a binary heap
    - Sorting: The Heap Sort algorithm sorted the array in both ascending and descending order
    - Extract Operations: Extracting the maximum or minimum element from the heap returned the correct value while maintaining heap properties
    - Insertion & Modification: The heap maintained its structure even after inserting new elements or modifying existing values



- Example Output:
  - Input:
    - Number of elements: 4
    - Elements: 2, 3, 5 ,7
    - Operation: Build Max Heap
  - Heap Output:
    - Max Heap structure: 7, 3, 5, 2
  - Sorting:
    - Chosen sorting order: Ascending (Heap Sort)
    - Extracted Maximum Value: 2
    - Remaining 7,3,5
  - These results confirm that the heap maintains its properties after operations such as sorting, extraction, and modification

- Encountered Problems
  - Heap Property Violation During Insertion
    - When inserting a new element into the heap, an issue arose where the heap property was violated because the element was not properly bubbled up to its correct position. This resulted in an incorrect heap structure, requiring an additional implementation of the heapify-up function to maintain order
  - Incorrect Indexing During Heapify Operations
    - During the max-heapify operation, an indexing issue caused the program to access an out-of-bounds array element, leading to unexpected behavior and incorrect heap structures. This was fixed by adding boundary condition checks to ensure that left and right child indices remained within the array limits.

- Conclusions
  - Binary heaps efficiently support priority queue operations
  - Max Heaps are useful when retrieving the largest element, while Min Heaps are used for the smallest element
  - Heap Sort is an effective sorting algorithm with a time complexity of O(n log n)
  - Implementing extraction and insertion while maintaining heap properties is crucual for correctness

- References
  - Lab 4 - Binary Heaps and Priority Queues